

QCon
全球软件开发大会

存量修复 + 增量拦截： B 站代码债务治理实践

何之真

哔哩哔哩 / 资深开发工程师



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

- 01 技术债治理的痛点与 AI 的价值
- 02 双轨治理：总体架构与核心原则
- 03 存量修复：持续偿还历史问题
- 04 增量拦截：把新问题拦在主干之外
- 05 总结与展望

01

技术债治理的痛点与 AI 的价值

千万行级代码库的现实



代码仓库持续增长，历史遗留 Bug 与安全漏洞不断累积

问题堆积



扫描平台里一直有大量历史告警，大家都知道，但排期永远让位于新需求

优先级低



做一次专项治理，告警曲线会短暂下降，然后迅速回升

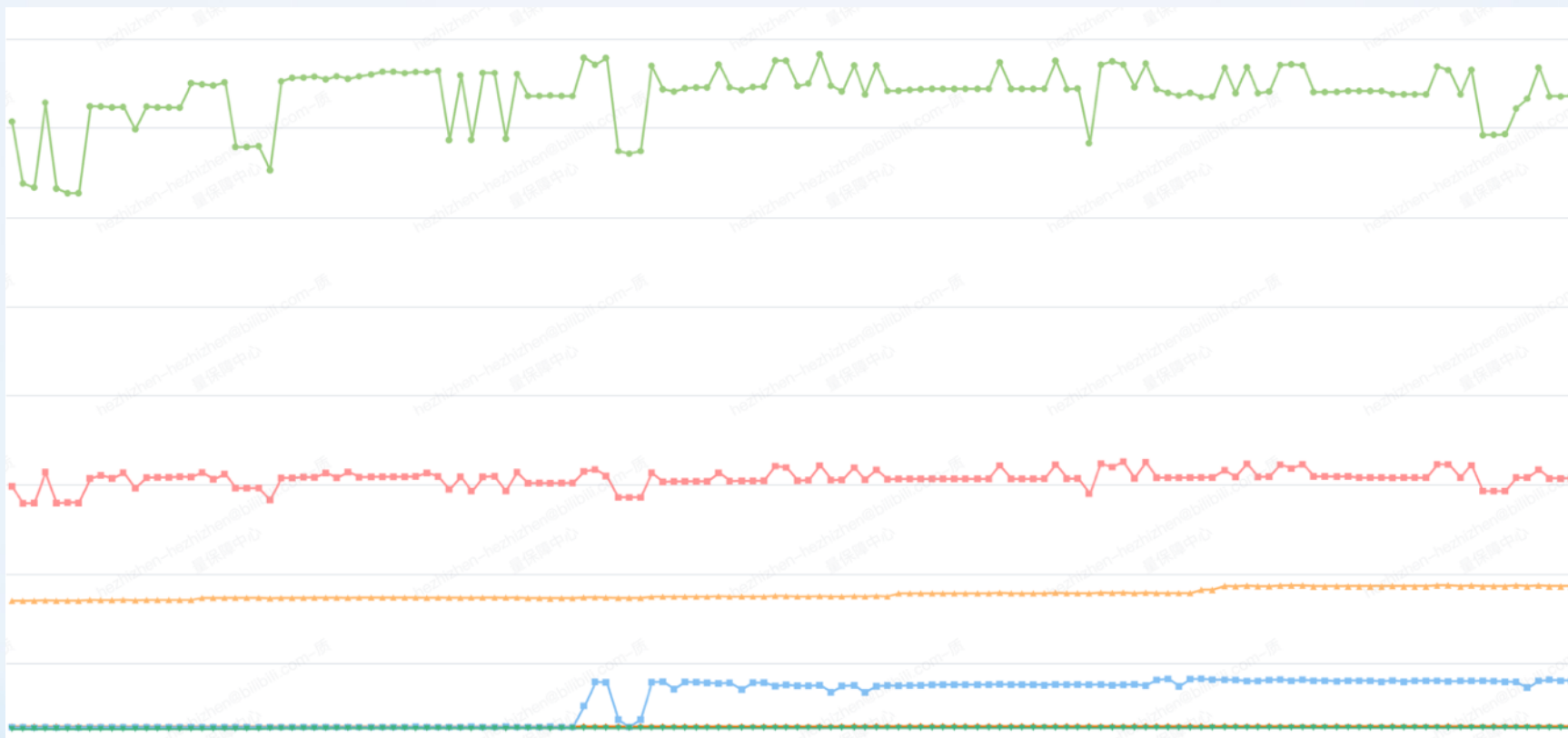
专项不治本



技术债不是没人发现，而是没有稳定的偿还机制

缺偿还机制

● 千万行级代码库，季度新增百万行



过去半年单个大部门下各子部门的代码量变化趋势

● 人工修复的困境



专项治理

每次专项治理投入大量人力，但效果只能维持几周



引入速度

新功能迭代不断引入新问题，速度往往超过修复速度



修复周期

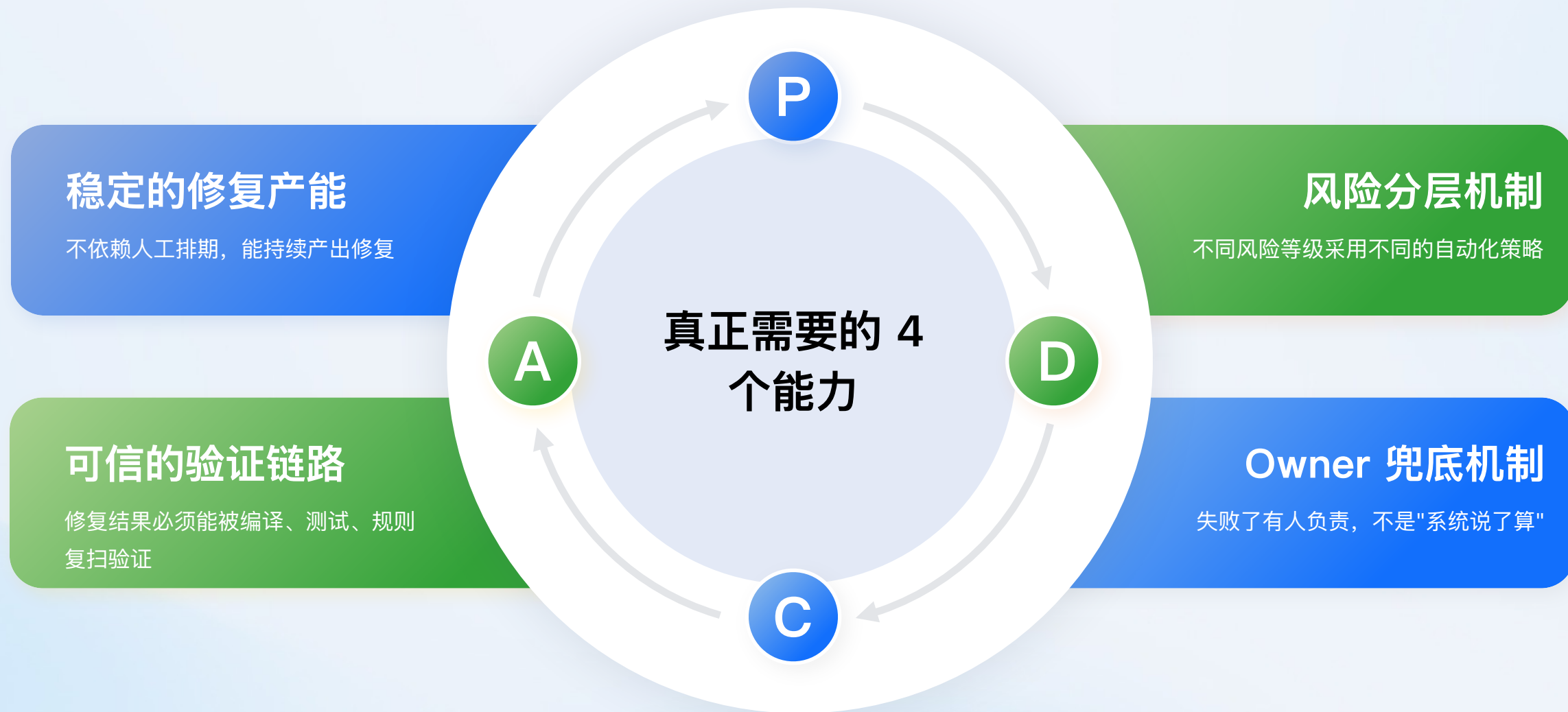
人工修复周期长：定位 → 理解 → 修复 → 测试 → Review → 合入



核心矛盾

修复是离散事件，但引入是持续过程

问题的本质：发现 ≠ 解决



AI 介入的前提与边界

1 规则、意图是否清晰

AI 更适合处理规则清晰、重复出现的问题，而设计意图不明确的问题更适合人工决策

2 上下文边界是否明确

AI 更适合处理上下文边界明确的问题，而涉及复杂业务语义取舍的问题更适合人工决策

AI 处理 VS 人工决策

3 能否通过编译、测试验证

AI 更适合处理能被编译 / 测试 / 静态分析验证的问题，而跨系统、跨服务的架构改造更适合人工决策

4 是否有 SOP

AI 更适合处理修复动作相对标准化的问题，而需要 owner 强判断的改动更适合人工决策

AI 在技术债治理中的价值与定位

AI 是工程体系中的一个生产力组件，需要被验证链路和风险分层"框住"才能发挥价值

规模化修复产能

把人工逐个修复变成批量自动生成 + 验证

增量实时拦截

在 MR 阶段实时识别风险，而不是事后扫描



知识沉淀与复用

修复经验沉淀为 Prompt 模板和评测集，持续优化

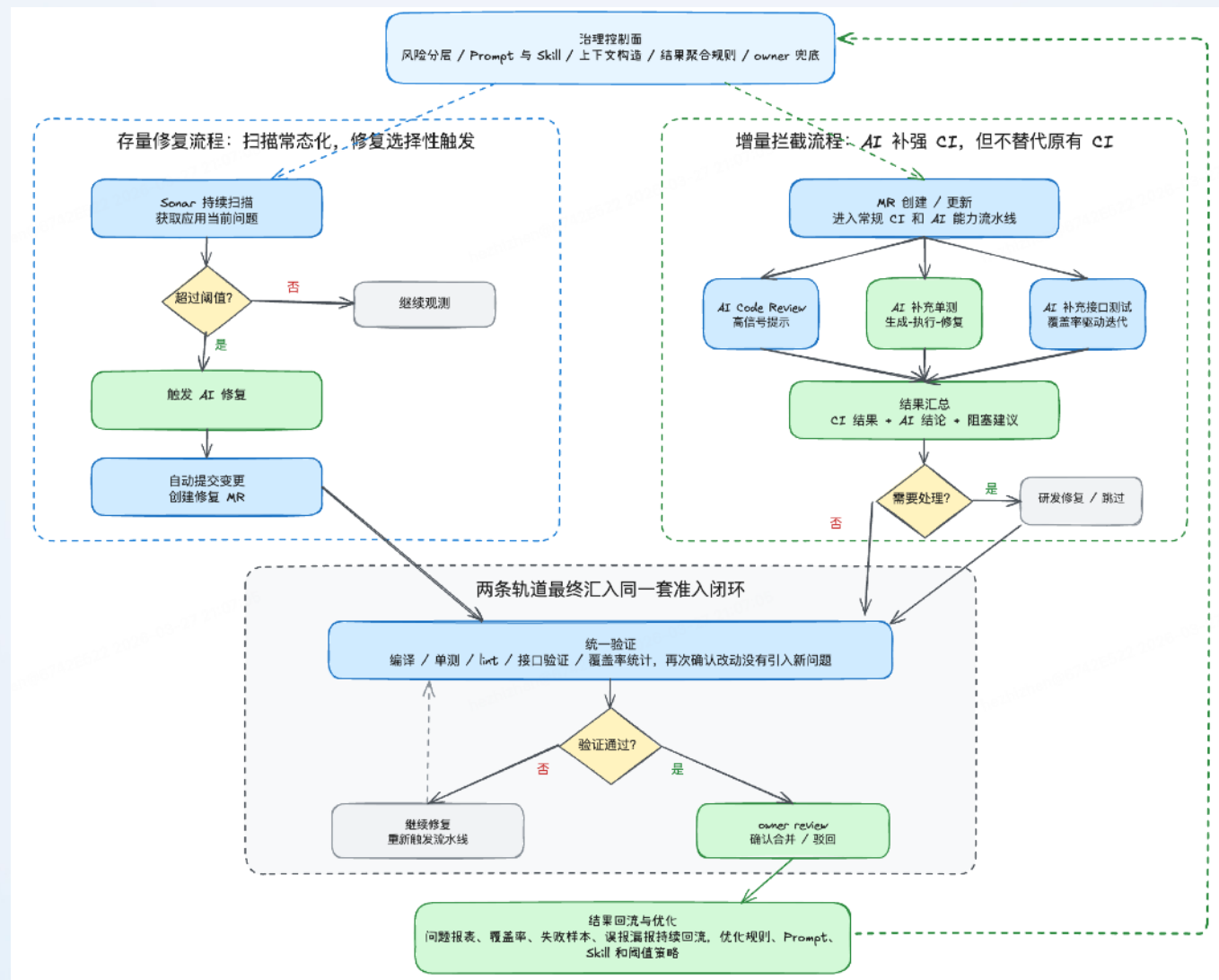
降低修复门槛

开发者不需要深入理解每条规则就能完成修复

02

双轨治理：总体架构与核心原则

总体架构



核心原则



双轨并进

存量修复和增量拦截必须同时推进，不能二选一

风险分层

自动化级别必须按照风险等级进行分层，而不是一刀切

职责明确

规则和变更检测负责发现问题，AI 负责针对规则清晰、边界明确且可验证的问题进行处理，流水线负责验证结果，owner 负责最终风险决策

回流优化

所有成功和失败结果都要回流，持续优化规则、Prompt 和策略

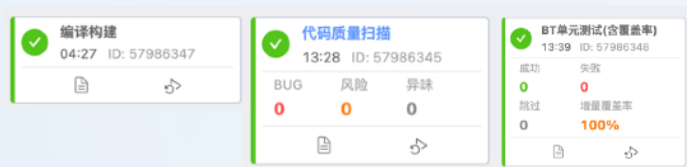
03

存量修复：持续偿还历史问题

从扫描到合入的全流程

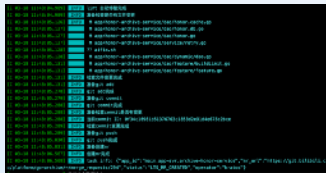
流水线验证

MR 创建后，会自动执行一系列如编译、单测、lint 等流水线来验证，等待应用 owner 来合并



AI 修复

当应用达到修复阈值时，触发自动修复的流水线，让 AI 使用 lint 工具并结合修复的要求进行全量修复，最后提交变更并创建 MR，通知应用 owner 去 review 与合并



Sonar 扫描

MR 合并后，会自动执行 Sonar 扫描，获取应用最新的问题数



对应用进行选择性修复

扫描常态化，修复选择性触发

持续扫描

每个应用在 MR 合并后都会自动跑 Sonar，获取最新的所有问题

owner 把关

应用的 owner 是最后一道门槛，需要人工的确认



报表汇总

按应用等级、部门、问题严重程度等不同维度进行汇总，直观看到债务的分布情况



阈值触发修复

当应用达到修复的阈值时，才会执行修复的流水线

✓ 运行状态	kratos kratos lint自动修复 持续时间 3 分钟 43 秒
--------	--

AI 的介入与约束

AI 修复循环

下载 prompt 作为修复的要求和约束，使用 lint 工具获取问题，逐项修复并再次使用 lint 工具验证，直至问题清零，最后提交变更创建 MR

Prompt 与 lint 工具

prompt 写清楚修复的详细步骤、规则策略、修改边界，工具用于发现问题同时验证 AI 的修复是否有效（解决了原有的问题，以及没有引入新问题）



AI 在约束下完成修复，并通过流水线验证

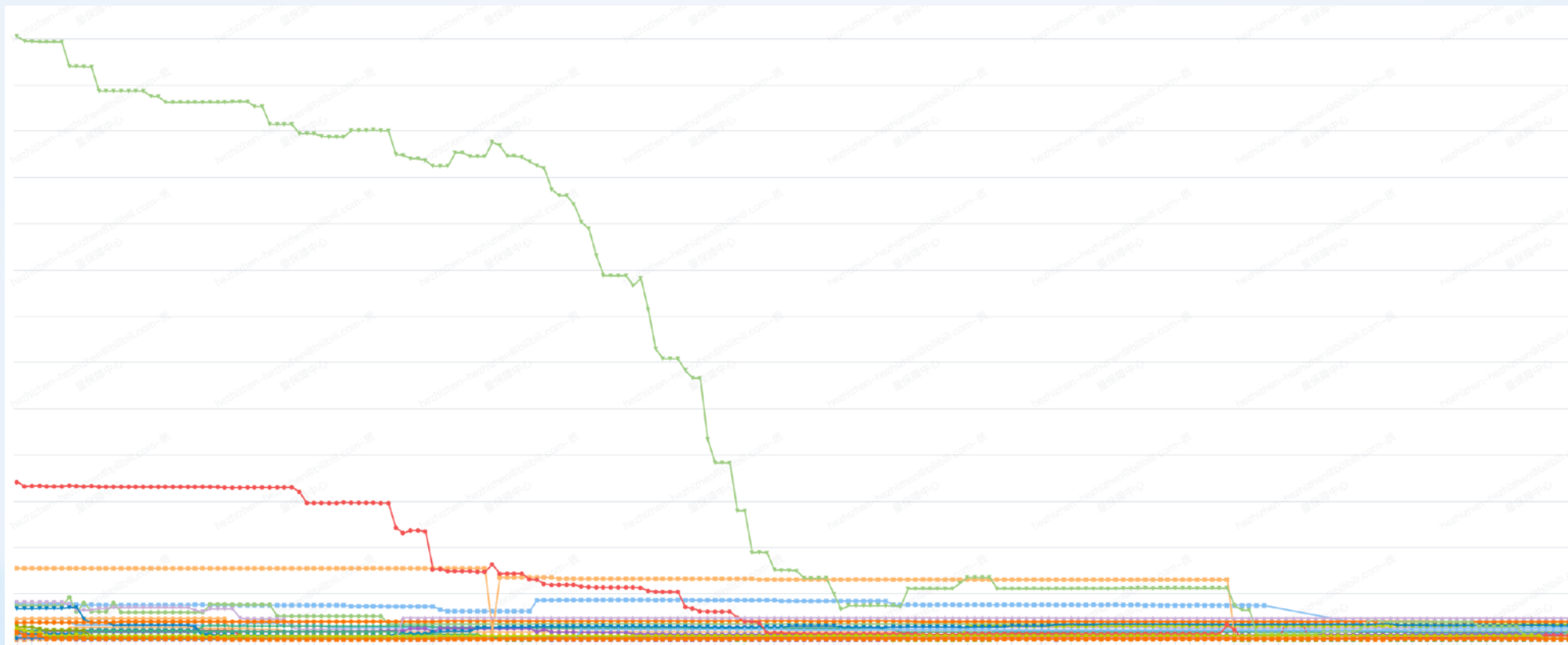
MR 流水线验证

MR 自动触发编译、单测、lint 等流水线验证 AI 的修复

人工最终决策

创建 MR 后会通知应用 owner，应用 owner 负责最终决策（是否合并）

● 存量修复的效果：季度下降 90%



过去半年内单个大部门下部分业务的代码异味的变化趋势

04

增量拦截：把新问题拦在主干之外

增量拦截整体链路

AI 并非替代原有 CI 流程，而是对它的补充和增强



AI Code Review: 传统 lint 的有效补充

1 从编写 linter 到构建自然语言规则

传统 linter 往往需要先把经验固化成明确规则，再交给工具执行；这意味着只有“能被形式化表达的问题”才能被稳定识别。AI Code Review 带来的变化是，可以直接用自然语言描述审查意图，将原本难以规则化的经验、规范和风险判断转化为可执行的审查能力，从而显著降低规则编写与维护成本。

2 结合代码上下文，输出高置信度审查结论

相比仅基于单行代码或局部模式匹配的检查方式，AI 可以结合函数、模块、调用链乃至变更意图理解代码。这样的上下文感知能力，使它更有机会识别真实问题、降低无效告警，并输出更接近人工审查判断的结论，让审查结果真正具备可用性和参考价值。

AI Code Review

3 从需求视角切入：代码审查的另一面

很多代码问题并不体现在语法、风格或局部实现上，而体现在“代码是否真正满足需求”。AI Code Review 不仅可以检查实现质量，还可以进一步对照需求描述、接口约束和预期行为，判断实现是否存在遗漏、偏差或误解。也就是说，代码审查不仅是看“写得对不对”，也是看“做得是不是用户真正想要的”。

4 构建数据闭环，形成持续演进的审查飞轮

AI Code Review 的效果并不是一次性确定的，而是可以在使用过程中持续优化。通过沉淀审查结果、人工反馈、误报分析和问题归因，可以不断迭代提示词、策略和样本数据，逐步提升问题识别能力与结论质量。随着数据积累与反馈回流，系统会形成一个持续增强的审查飞轮。



AI Code Review: 自然语言维护的审查规则

规则类型	规则代码	通用语言	是否拦截	规则详情
应用配置	GO-CONFIG-001	golang	是	[] 热更新配置时, 必须反序列化到新创建的零值对象, 再原子替换指针。禁止对已有配置对象重复 Unmarshal, 会导致 slice 内 struct 字段 merge 而非覆盖。 ✗ 警惕模式: 加锁+原地更新 (merge 问题依然存在) <code>go mu.Lock() toml.Decode(newConfig, &existingConf) // merge, 旧字段值被保留 mu.Unlock()</code> ✓ 安全模式: 每次创建对象, 然后原子替换 <code>go var newConf Config toml.Decode(newToml, &newConf) // 新对象, 零值开始, 不会 merge mu.Lock() conf = newConf // 整体替换 mu.Unlock()</code> – 更优: 用 <code>atomic.Pointer</code> (无锁读) <code>go var confPtr atomic.Pointer[Config] newConf := &Config{} toml.Decode(newToml, newConf) confPtr.Store(newConf) ```</code>
panic	GO-PANIC-002	golang	是	[] 是否忽略错误后继续使用可能为空的指针? ✗ 警惕模式: <code>user, _ := getUser(id); return user.Name // 忽略err直接用user</code> ✓ 安全模式: <code>user, err := getUser(id); if err != nil { return err }</code> ⚠ 返回值必须包含err并忽略才满足此规则, 如无err则不满足此规则
数据库慢查询	GO-DATABASE-006	golang	是	[] 是否在循环中开启/提交事务? ✗ 警惕模式: <code>for { tx.Begin(); ...; tx.Commit() }</code> ✓ 安全模式: 在循环外开启一个大事务
数据库慢查询	GO-DATABASE-007	golang	是	[] 长事务是否会锁表? ✗ 警惕模式: <code>UPDATE large_table SET field = ? 未分批</code> ✓ 安全模式: 分批更新, 每批1000-5000条)
数据竞争Race	GO-DATABASE-001	golang	是	[] 是否在goroutine内部修改外部的map/slice/变量, 且无同步保护? ✗ 警惕模式: <code>go go func() { sharedMap[key] = value }() // 并发写 go func() { sharedSlice = append(sharedSlice, item) }() // 并发append go func() { counter++ }() // 并发修改变量</code> ✓ 安全模式: <code>go mu.Lock() sharedMap[key] = value mu.Unlock() // 或使用 sync.Map, atomic包</code> ⚠ 注意: 仅读取不会race, 但读+写并发会race
数据竞争Race	GO-DATABASE-003	golang	是	[] 是否在goroutine中直接使用循环迭代变量? (Go < 1.22) ✗ 警惕模式: <code>for _, item := range items { go func() { use(item) }() }</code> ✓ 安全模式: <code>for _, item := range items { item := item; go func() { use(item) }() }</code> ⚠ Go 1.22+已修复此问题
数据竞争Race	GO-DATABASE-004	golang	是	[] 是否对sync.WaitGroup、sync.Mutex等使用了值拷贝? ✗ 警惕模式: <code>func process(wg sync.WaitGroup) // 值传递会拷贝</code> ✓ 安全模式: <code>func process(wg *sync.WaitGroup) // 指针传递)</code>
panic	GO-PANIC-001	golang	是	[] 是否存在未检查的类型断言? ✗ 警惕模式: <code>val := data.(string) // 直接断言, 类型不匹配会panic</code> ✓ 安全模式: <code>val, ok := data.(string); if !ok { return err }</code>



AI Code Review: 增量拦截的分层处理

agile @agile · 2 weeks ago

Maintainer

AI CodeReview 报告

原advisor AI Review功能已迁移至agileflow, 功能介绍 [info](#), 有任何需求/问题请联系@火羽白

✅ 审查通过, 剩余 0 个问题待处理

1. 审查任务进度(0aacb8f9) 【进度查看】

初始化

重点问题

建议问题(未开启)

总体评价

2. 审查结果

▼ 重点关注问题 【规则查看】

文件	问题描述	严重级别	命中规则	状态
app/service/pantheon/internal/dao/client.tx.go	URL解析错误后仍使用	高危	GO-PANIC-002	✅ 已解决
app/service/pantheon/internal/dao/client.tx.go	成功日志打印完整请求体	中危	GO-RESOURCE-004	✅ 已解决
app/service/pantheon/internal/service/common/company/tencent.go	忽略错误后继续解引用	高危	GO-PANIC-002	✅ 已解决
app/service/pantheon/internal/service/common/company/tencent.go	高频路径打印完整参数	中危	GO-RESOURCE-004	✅ 已解决

3. 总体评价

需求实现

✅ 新增米大师token查询接口

- ✅ 开放平台新增鉴权入口
 - bind.bm.go:LiveLinkMidasToken > HTTP 路由定义
 - server.go:New > PathBindLiveLinkMidasToken 鉴权注入
- ✅ 调用米大师并返回token
 - service.bind.v2.go:LiveLinkMidasToken > bindLink 解析与 Pantheon 调用

agile @agile started a thread on an old version of the diff 2 weeks ago

Resolved by agile 2 weeks ago

Hide thread

app/service/pantheon/internal/service/common/company/tencent.go

```
429 +     if err != nil {
430 +         log.Errorf(ctx, "[TxLiveLinkMidasToken] call TxLiveLinkStoreUrlParams err, req:%+v,
      err:%+v ", req, err)
431 +         return
432 +     }
433 +     actIdstr := strconv.FormatInt(storeUrlParamsResp.ActId, 10)
434 +
435 +     extraReq := model.TxBindInfoExtraReq{
436 +         TxActID: actIdstr,
437 +         TxScene: "",
438 +     }
439 +     bindAddress, err := s.GetBindAddress(ctx, &v1pb.GetBindAddressReq{
440 +         BizCode: req.BizCode,
441 +         Uid:      req.Uid,
442 +         ExtraReq: utils.JsonMarshalIgnoreErr(extraReq),
443 +     }, conf)
444 +     if err != nil {
```

agile @agile · 2 weeks ago

Maintainer

▼ 【✅ 已解决】忽略错误后继续解引用 (高危) (GO-PANIC-002)

问题描述

- 具体问题点及位置: 在 tencent.go 第444-453行, GetBindAddress 返回 err 后仅记录日志未返回, 随后直接访问 bindAddress.Livlink.Sign/Code/TimeStamp 等字段。
- 影响范围: 当 GetBindAddress 失败或返回的 Livlink 为空时, 该链路会在生成 params 时触发空指针panic, 影响 TxLiveLinkMidasToken 接口稳定性。
- 风险评估: 高风险, 线上请求在异常配置或下游失败场景下可直接崩溃当前请求处理流程。

修复建议

GetBindAddress 返回错误时应立即返回; 并在使用 bindAddress.Livlink 前增加 nil 校验, 避免对空指针取字段。

Edited by agile 2 weeks ago

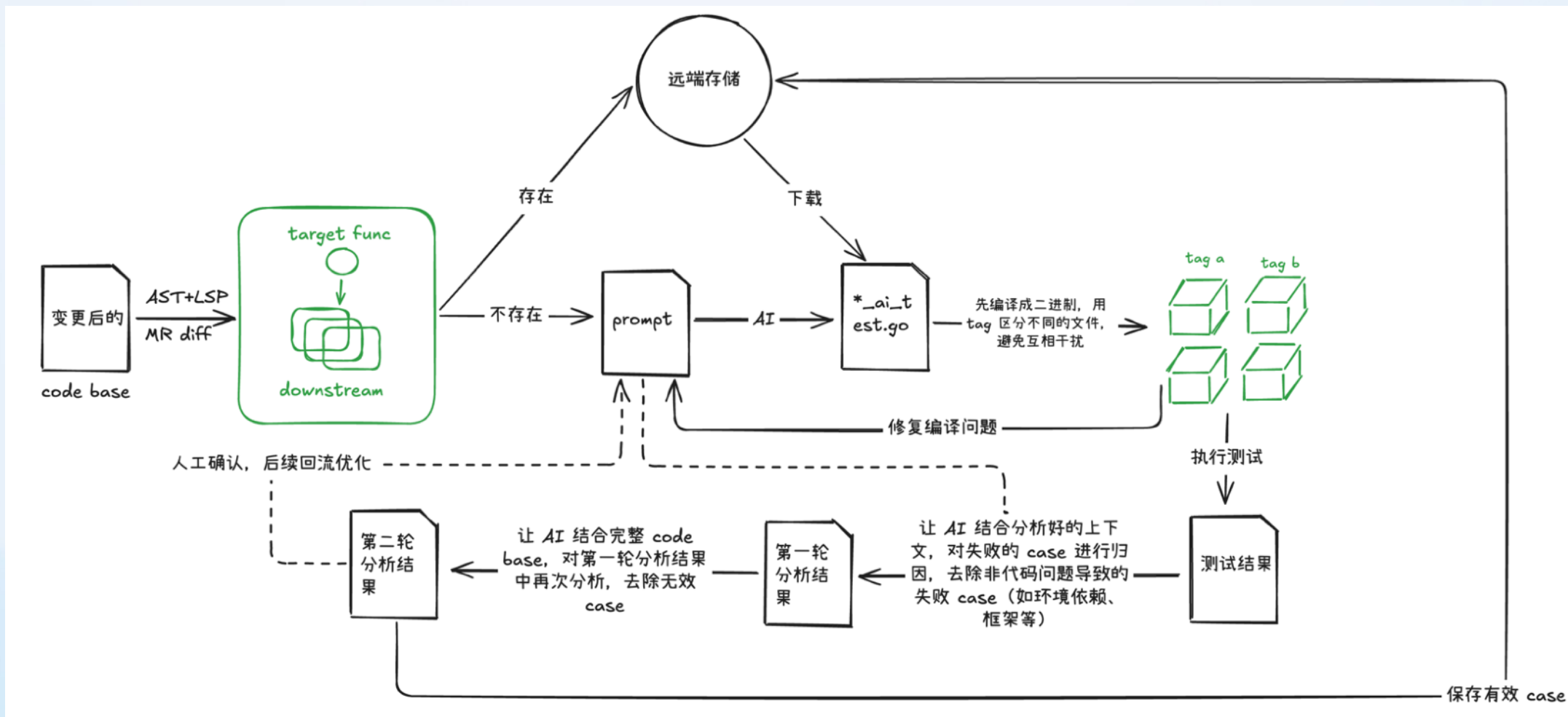


AI Code Review: 持续迭代与优化

MR总数	问题总数	误报数	暂不处理数	已解决数	准确率
281	1,124	168	547	387	85 %

描述	仓库	文件	commit_sha	MRID	类型	问题状态	核验状态
+ 修复备份字节未判空	operation-data-plat...	OpenclawDomain...	89f9320c	135	空指针	待解决	● 未复核 ▾
+ 异常日志链式调用NPE	operation-data-plat...	OpenclawDomain...	89f9320c	135	空指针	已解决	● 未复核 ▾
硬编码SYSTEM绕过鉴权	game-providence-s...	OrderCommandS...	cce25396	11	安全	待解决	● 未复核 ▾
+ 外部结果可能返回null	go-bdjs	processor_archiv...	dfa27716	19673	结构体字段变更	误判	● 未复核 ▾
+ 外部结果字段可能由{}变null	go-bdjs	processor_archiv...	9fd71df3	19673	结构体字段变更	待解决	● 未复核 ▾
+ 未检查的类型断言	go-ott	view_history.go	e223dd79	7487	panic	已解决	● 未复核 ▾
+ 未检查的类型断言	go-ott	view_history.go	a603119d	7487	panic	已解决	● 未复核 ▾
+ 高频路径Info日志	go-operational	guess.go	766095ea	3082	资源泄露	已解决	● 未复核 ▾
+ 循环内错误日志可能刷屏	go-operational	reserve_countersg...	766095ea	3082	资源泄露	已解决	● 未复核 ▾

AI 补充单元测试：围绕 MR 变更函数的自动生成与修复





<pre> 441 func TestBuildCommonPgcCardService_PaymentPayCardNilValueRisk(t *testing.T) { 442 monkey.PatchConvey("Payment.PayCard nil value causes panic", t, func() { 443 // 设置mocks 444 monkey.Mock(smoke1.ConvertSnIdToLong).Return(int64(777)).Build() 445 monkey.Mock(common.GetHttps).Do(func(v string) string { return v }).Build() 446 monkey.Mock(common.FromMarsalIPTV).Return(false).Build() 447 monkey.Mock(track.Debug).Return().Build() 448 monkey.Mock(smoke1.HitStatusGate).Return(false).Build() 449 monkey.Mock(spk.BuildPgcdList).Return(pgcdList{HorizontalUrl: "h"}).Build() 450 monkey.Mock(tc.GetService).Do(func(pgSeed) bool { return false }).Build() 451 monkey.Mock(cmdoas.Dao).HitSns().Return(false).Build() 452 monkey.Mock(cmdoas.Dao).SetPgcpInfo().Return(map[string]string{}).Build() 453 monkey.Mock(cmdoas.Dao).CommentSpisootMarks().Return().Build() 454 monkey.Mock(cmdoas.Dao).CanReplaceCornerMark().Return(false).Build() 455 monkey.Mock(yuan2cent).Return("2").Build() 456 monkey.Mock(tc.GetService).pgcPaymen(Desc).Return("").Build() 457 monkey.Mock(spk.PaymentShowExt).Return(spk.PaymentShowExt{}).Build() 458 monkey.Mock(common.getPglLabel).Return().Build() 459 460 b := &service{ 461 conf: &conf.Config{ 462 </pre>	执行结果 失败 存在风险 直接原因：在调用 Payment.PayCard 时未对 map value 判空，即可直接读取 value.Title 字段。若某个 errory 的 value 为 nil 会触发空指针；当前则在 common_card.go:960 处处理。关键代码逻辑： <pre> app/domain/card/service/app/common_card.go:960 [? ?] value' 的直接解引用) : 上述数据路径: pgcViewSeason 通过 JSON 方式转换为 CommonSeasonV2 (app/domain/card/service/pgccard_transform.go:60的 ConvertPgcSeasonToLocalSeason)，过程中不检查该 PayCard；且名称 OTT 服务仅在 app/domain/service/internal/mode/copy.go:17 调用 map 中的 nil 值 (DeepCopySeasonPayCard 对 nil 返回 nil)，因此其类上可析出 value=nil 的 PayCard 条目，成立前提：Payment.PayCard 长度>0 且其中至少一值 value 为 nil。 </pre> ☒ 有效 case ☐ 无效 case 运行前修改于 11天前 编辑 人工确认 下载数据，不能完全信任
---	---

不仅生成单测，还把生成结果变成可审计、可量化、可回看的报告



AI 补充单元测试：试点效果展示

MR总数

145

变更函数数量

2,580

覆盖率

84.9%

Case总数

13,719

成功Case数

13,246

失败Case数

348

异常Case数

125

AI标注有效Case

28

人工标注有效Case

12

人工标注无效Case

10

正确率

54.5%

AI 补充接口测试：覆盖率驱动的迭代闭环

围绕覆盖率盲区自动生成测试用例，使用基于 mock 的回放工具通过回放发现缺失 mock，并按协议逐条修复，持续提升目标接口覆盖率。

构造测试用例

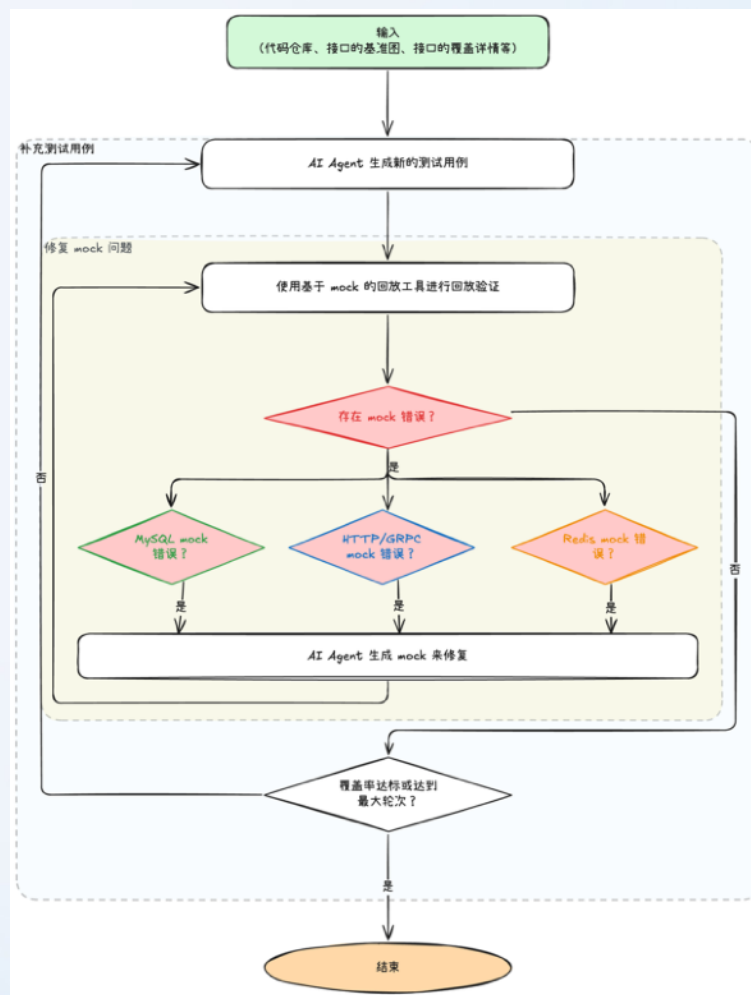
结合接口的源码、调用链路和覆盖详情，让 AI 设计测试用例以最大程度覆盖当前未覆盖的链路，明确测试场景、输入参数、预期响应和预期下游调用（包含 MySQL、Redis、HTTP、gRPC 等）。

持续回放验证与 mock 修复

先通过回放执行，发现缺失 mock，按顺序修复，一次只修一条（避免级联问题）；再持续通过回放执行来验证修复是否正确，如果有问题，则基于源码、预期等内容修改 mock，或 bypass 带有时间戳、随机值等内容的下游调用

数据链路复盘与差异对比

当 mock 问题均已修复，对比接口的实际响应与预期响应是否一致，如果不一致则根据日志进行数据链路复盘，对比整体接口和每一处下游调用是否一致，最后交由人工确认





AI 补充接口测试：试点效果展示

场景：StartLive happy path - 合法参数开播 (android 平台, 竖屏, 普通直播类型)

请求参数：

```
method: POST
path: /live.streaming.Streaming/StartLive
headers:
  :authority: 127.0.0.1:9000
  :method: POST
  :path: /live.streaming.Streaming/StartLive
  :scheme: http
  content-type: application/grpc
  te: trailers
body (protoscope):
  1: 10001
  2: {"android"}
  3: 33
  15: {1: 10001 2: {"192.168.1.1"}}
```

预期响应：

```
status_code: 200
grpc-status: 0
body: 包含 stat / live_key / rtmp 等字段
```

预期下游调用：

- [Redis] fake_command 查询 → 返回不需要 fake
- [MySQL] room_status 查询 → 返回房间状态
- [gRPC] room/member 聚合查询 → 返回房间和用户信息
- [MySQL] live_record/flow_record 写入 → 成功

```
StartLive (streaming.go:L38-L60) - 覆盖率 50%
├── fakeCommand.SceneNeedFakeCommand - HIT
├── facadeStartLive.BlinkStartLive (facade_start_live.go:L54-L120) - 覆盖率 18%
│   ├── FixRawParams - HIT (通过)
│   ├── BuildStartLiveLocatorFromRequest - HIT
│   ├── RawParamsCheck - HIT (通过)
│   └── applicationStartLive.BlinkStartLive (application_start_live.go:L48-L144) - 覆盖率 14%
│       ├── loadRoomAggregateToLoc - HIT (成功)
│       │   └── GetRoomAggregate - HIT (成功, 返回 uid=8522261)
│       ├── loadStreamMngToLoc - HIT
│       ├── startLiveRoomStatusCheck - HIT (失败! uid 不匹配)
│       │   └── IsRoomBelongToMid(10001) ≠ 8522261 → ErrUnauthorized
│       ├── 未覆盖: loadMemberAggregateToLoc (L69)
│       ├── 未覆盖: checkSpecialTypeRoom (L77)
│       ├── 未覆盖: startLiveLock (L105)
│       ├── 未覆盖: BlinkStartLiveCheck (L119) - 149 uncov lines
│       ├── 未覆盖: StartLiveGetUpStreamInfo (L126)
│       ├── 未覆盖: blinkStartLiveCommand (L139) - 59 uncov lines
│       └── 未覆盖: setRoomStatusStartLive - 59 uncov lines
└── BuildStartLiveResponse - HIT (错误处理)
    └── 未覆盖分支: resp=nil && err=nil (L42-44)
```

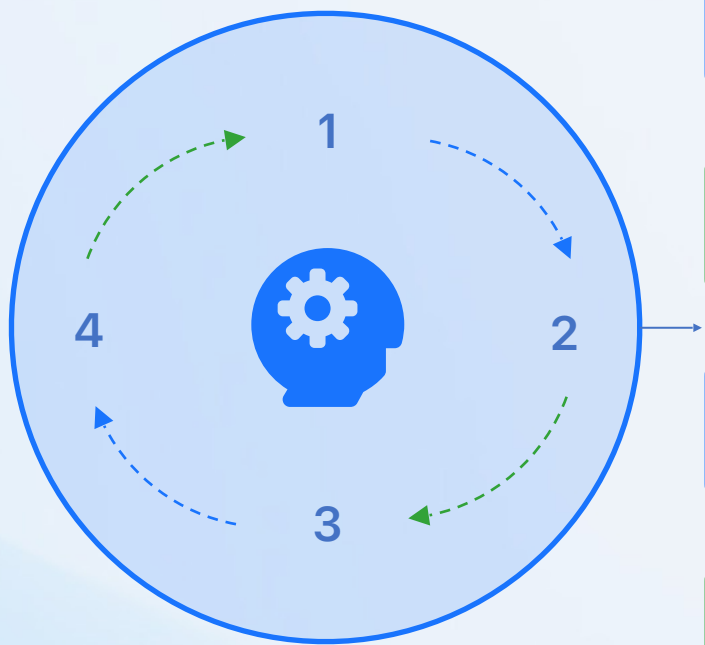
关键未覆盖节点 (按 uncovered_line 降序)：

1. BlinkStartLiveCheck - 149 uncov lines
2. MemberCheck - 72 uncov lines
3. BlinkStartLive(app) - 62 uncov lines
4. GaiaRiskCheck - 62 uncov lines
5. RoomCheck - 62 uncov lines
6. blinkStartLiveCommand - 59 uncov lines
7. setRoomStatusStartLive - 59 uncov lines
8. Acquire (distributed_lock) - 51 uncov lines

05

总结与展望

总结



持续运行

双轨并进

验证优先

先建立信任

技术债治理不是一次专项，而是一条持续运行的工程流水线。

存量修复负责偿还旧账，增量拦截负责止损，两条轨道缺一不可。

真正决定上限的不是模型，而是问题选择、风险边界和验证链路。

AI 只有进入工程系统并被验证、分层、兜底，才会变成稳定生产力。

局限与挑战

当前局限

- 存量修复目前仅覆盖部分低风险规则类型
- AI 补充单测和接口测试仍处于小规模试点阶段

现实挑战

- MR 合并效率仍影响自动修复闭环的最终效果
- 覆盖率提升依赖稳定环境、mock 质量和运行链路
- 模型成本、上下文长度和执行性能仍需持续优化

落地方法论

先证明局部成立，再做平台化扩张

阶段一

选一类问题

规则清晰、验证充分、风险低，
是最好的起点。

阶段二

跑通局部闭环

在少量仓库验证扫描、修复、验证、合入的完整流程。

阶段三

双轨打通

把增量拦截接入 MR 流程，形成
“偿还 + 止损”的联动机制。

阶段四

平台化扩张

逐步扩大规则覆盖、仓库范围和
自动化级别。

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

